

动态开点线段树

线段树需要开4倍空间，所以有的时候比较耗费空间，因为他的编号并不是连续的，所以实际上线段树也是可以动态开点的，方法和01trie比较类似，新建一个节点对这个节点开一个编号。

但是这样编号为 id 的节点的左右儿子就不是 $id*2$ 和 $id*2+1$ 了，所以我们还在要在结构体内加两个变量分别记录左右儿子的编号。

总体来说，动态开点空间比常规线段树少一半，甚至1/4都可以。而且对于数据很大，但是操作很少的题，这是常规线段树解决不了的。

动态开点线段树

```
int build(int l,int r)
{
    ++k;//k表示当前节点编号
    if(l==r)
    {
        tree[id].sum=a[l];
        return k;//把当前节点编号返回回去
    }
    int mid=(l+r)/2;
    tree[k].lson=build(l,mid);
    tree[k].rson=build(mid+1,r);
    tree[k].sum=tree[tree[k].lson].sum+tree[tree[k].rson].sum;
    return k;//把节点编号返回去
}
root=build(1,n);
```

线段树

实际上线段树还有一种动态开点的办法，一般的线段树都是在最开始的时候就把线段树每个节点建好，然后再去修改查询，这里介绍一种最开始不建线段树的做法，最开始只有一个节点，如果有一次修改操作。比如单点修改，那么我就只对需要查询到的点新增节点，其他访问不到的点我不用去建节点，那么每次修改的时间辅助就是 $\log n$ 。同样访问也是一样，我要去查询这个区间，我只把需要用到的点建出来，其他点不管，最坏时间复杂为 $\log n^2$. 和线段树点的区间没有关系。

但是这种线段树只能适用于知道最开始线段树所有节点信息的题

线段树

一：最开始

①

$[1, 10]$

二：修改编号为k的节点，权值为x

比如 3 5

①

$[1, 10]$

②

$[1, 5]$

③

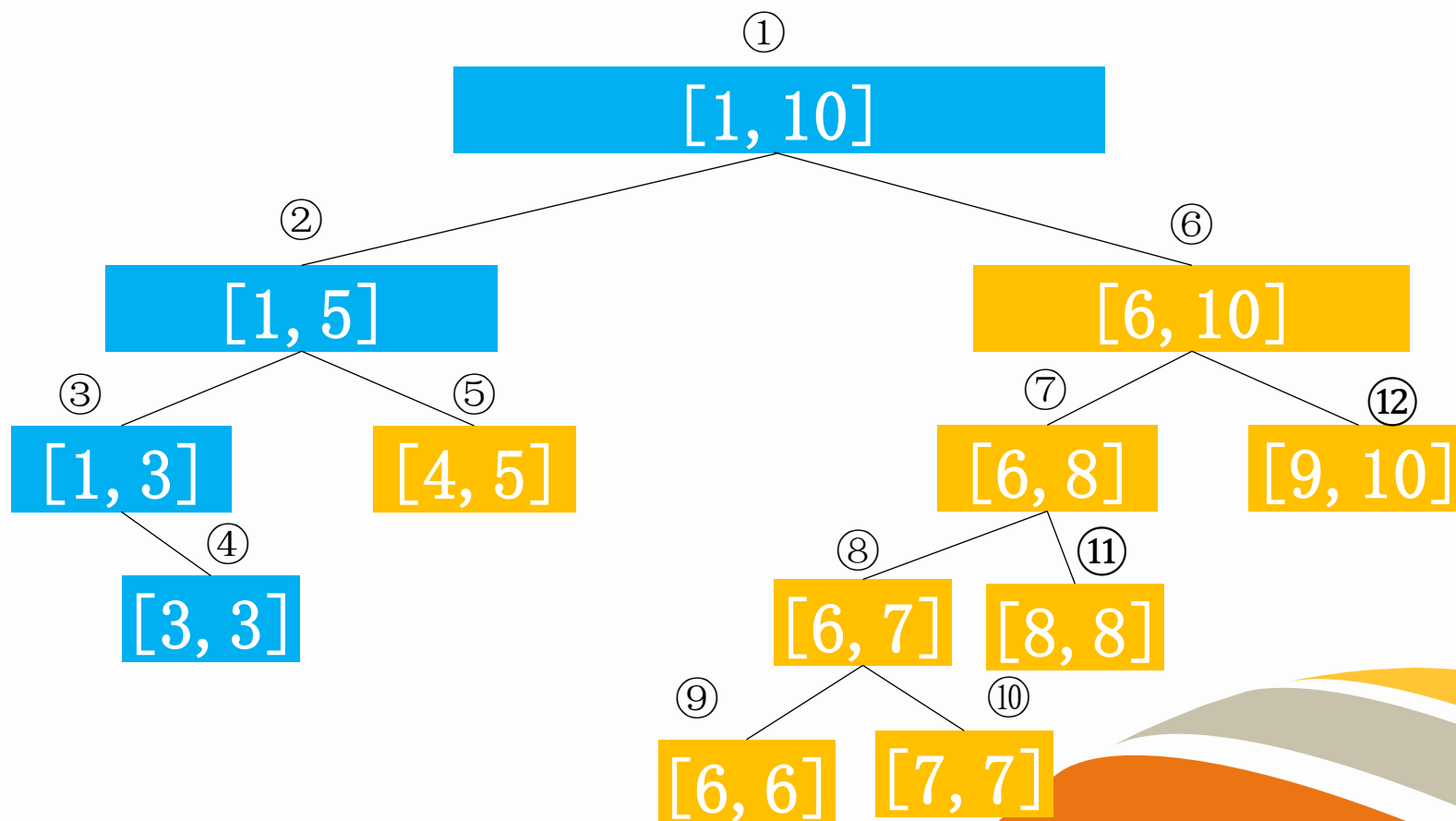
$[1, 3]$

④

$[3, 3]$

线段树

三：查询区间 $[2, 7]$ 的最大值



动态开点线段树

线段树建树（只建根结点）

动态开点线段树仅能用于初始值为空或者初始值都相同的情况。

```
int build(int l,int r)
{
    ++k;//当前结点编号为k
    tree[k].l=l;
    tree[k].r=r;
    return k;
}
root=build(1,n);
```

动态开点线段树

单点更新，如果要去下一层，少哪个儿子先把这个儿子创建出来

```
void update(int id,int ki,int x)
{
    if(tree[id].l==tree[id].r)//如果找到答案，返回
    {
        tree[id].num=x;
        return ;
    }//如果要去下一层找，那么先判断左右儿子是否存在
    int mid=(tree[id].l+tree[id].r)/2;
    if(!tree[id].ls)
        tree[id].ls=build(l,mid);
    if(!tree[id].rs)
        tree[id].rs=build(mid+1,r);
    if(ki<=mid)
        update(tree[id].ls,ki,x);
    else
        update(tree[id].rs,ki,x);
    ...
}
update(1,i,x);//单点修改
```

动态开点线段树

区间修改，和单点修改做法类似，一定要注意先创建儿子再下传标记

```
void update(int id,int l,int r,int x)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        ...
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(!tree[id].ls)
        tree[id].ls=build(l,mid);
    if(!tree[id].rs)
        tree[id].rs=build(mid+1,r);
    if(tree[id].lazy)
        pushdown(id); // 一定是先建子节点再下传标记
    ...
    //后面一样的写法
}
update(1,l,r,x);
```




动态开点线段树

区间查询，和区间修改一样。



动态开点线段树

动态开点线段树还是特别重要的，他只创建会被使用的点，其他没有被操作的点就直接当做一个整体考虑。虽然他操作比较繁琐，常数也比较大，但是他能实现很多普通线段树不能实现的操作。

- 1、不需要离散化就可以做权值线段树的题
- 2、如果题目是一个二维矩阵，每次修改一行中部分值，然后再询问某一行的信息。
- 3、给定 n 棵线段树，每次对一棵线段树进行操作
- 4、线段树合并，主席树。

列队

题目描述

展开

Sylvia 是一个热爱学习的女孩子。

前段时间, Sylvia 参加了学校的军训。众所周知, 军训的时候需要站方阵。

Sylvia 所在的方阵中有 $n \times m$ 名学生, 方阵的行数为 n , 列数为 m 。

为了便于管理, 教官在训练开始时, 按照从前到后, 从左到右的顺序给方阵中的学生从 1 到 $n \times m$ 编上了号码 (参见后面的样例)。即: 初始时, 第 i 行第 j 列的学生的编号是 $(i - 1) \times m + j$ 。

然而在练习方阵的时候, 经常会有学生因为各种各样的事情需要离队。在一天中, 一共发生了 q 件这样的离队事件。每一次离队事件可以用数对 (x, y) ($1 \leq x \leq n, 1 \leq y \leq m$) 描述, 表示第 x 行第 y 列的学生离队。

在有学生离队后, 队伍中出现了一个空位。为了队伍的整齐, 教官会依次下达这样的两条指令:

1. 向左看齐。这时第一列保持不动, 所有学生向左填补空缺。不难发现在这条指令之后, 空位在第 x 行第 m 列。
2. 向前看齐。这时第一行保持不动, 所有学生向前填补空缺。不难发现在这条指令之后, 空位在第 n 行第 m 列。

教官规定不能有两个或更多学生同时离队。即在前一个离队的学生归队之后, 下一个学生才能离队。因此在每一个离队的学生要归队时, 队伍中有且仅有第 n 行第 m 列一个空位, 这时这个学生会自然地填补到这个位置。

因为站方阵真的很无聊, 所以 Sylvia 想要计算每一次离队事件中, 离队的同学的编号是多少。

注意: 每一个同学的编号不会随着离队事件的发生而改变, 在发生离队事件后方阵中同学的编号可能是乱序的。

列队

我们先思考，如果只是对一行进行操作，我们应该如何去处理？

虽然设计到点的删除和插入，但是前面我们已经讲过，线段树的点删除是可以直接做的，维护一个真实儿子数量就可以了。这里的插入也都是在末尾插入，只要末尾有空位就行，那么有 n 次操作，最多再末尾插入 n 个元素，所以我只需要开 $2*n$ 的区间就能满足所有需要，只是初始是后面 n 个结点为空。

我们建一颗 $[1, 2n]$ 的线段树，初始时 $[1, n]$ 为有效结点， $[n+1, 2n]$ 为失效结点。

列队

但是该题有行和列，每次出队的同学可能在任意一行，所以我们需要维护 n 棵行平衡树，每棵平衡树表示区间 $[1, 2m]$ ，很明显初始化的时候空间就炸了。

但是最多只有 n 个同学出队，所以后面操作的空间特别小，很多节点根本没有被使用到，完全是浪费。

所以我们可以考虑动态开点线段树，初始时的 n 棵线段树只是 n 个根结点，来一次修改，修改一条链的信息，最多 n 次修改，那么最多存在 $2 * n * \log n$ 个结点。

同时，注意，还是列操作，但是列操作只会在最后一列，所以我们对最后一列再维护一颗列线段树，行线段树编号是连续的，列线段树编号不连续，所以先就建好。

列队

因为操作涉及行变化和最后一列的变化，所以对每一列的前 $m-1$ 个元素分别开 n 棵线段树，对最后一列单独开一棵线段树。我们可以把矩阵看做编号 $1—n*m$ ，所行线段树的区间就很好表示。列线段树中的点不是连续的，我们可以用 $1—n$ 表示，但是要记录真实点编号为 $m, 2*m \cdots n*m$

```
int build(long long l, long long r)
{
    k++;
    tree[k].l = l;
    tree[k].r = r;
    tree[k].son = r - l + 1;
    return k;
}
for (int i = 1; i <= n; i++)
    root[i] = build((i - 1) * m + 1, i * m - 1);
root[n + 1] = build(1, n);
```

列队

如果离开这个 $[x, y]$ 这个人不在最后一列，那么先对编号为 x 的线段树真实的第 y 个儿子做删除操作，再对列线段树真实的第 x 个儿子做删除操作，并且记录下来把这个点放入该行线段树的末尾去，最后把删除的这个结点放到列线段树的末尾去

```
int t1=del(root[x],y);//记录删除点的编号
int t2=del(root[n+1],x);
add(root[x],m,t1);//放入第x棵线段树末尾
add(root[n+1],n,t2);//放入列线段树末尾
```

如果在最后一列，那就更简单了，只需要在列线段树中做操作，完全不影响行线段树。

方伯伯的OJ

方伯伯正在做他的Oj。现在他在处理Oj上的用户排名问题。Oj上注册了 n 个用户，编号为 $1 \sim n$ ，一开始他们按照编号排名。

方伯伯会按照心情对这些用户做以下四种操作，修改用户的排名和编号：

1. 操作格式为 $1\ x\ y$ ，意味着将编号为 x 的用户编号改为 y ，而排名不变，执行完该操作后需要输出该用户在队列中的位置，数据保证 x 必然出现在队列中，同时， y 是一个当前不在排名中的编号。
2. 操作格式为 $2\ x$ ，意味着将编号为 x 的用户的排名提升到第一位，执行完该操作后需要输出执行该操作前编号为 x 用户的排名。
3. 操作格式为 $3\ x$ ，意味着将编号为 x 的用户的排名降到最后一位，执行完该操作后需要输出执行该操作前编号为 x 用户的排名。
4. 操作格式为 $4\ k$ ，意味着查询当前排名为 k 的用户编号，执行完该操作后需要输出当前操作用户的编号。

对于 100% 的数据， $1 \leq n \leq 10^8$ ， $1 \leq m \leq 10^5$

方伯伯的OJ

该题有针对关于编号的操作，也有针对关于排名的操作，所以无论是根据编号还是排名建树都不对，而且每次询问的编号或排名是分开的，所以我们也没有必要去写树套树。

所以我们可以考虑开两棵树，一棵根据排名建树，一棵根据编号建树。

对于排名线段树，记录每个排名所对应的用户的编号。对于编号线段树，记录该编号的人在线段树中的位置，对于编号线段树，他主要是起一个映射的作用，因为每次有人位置变化，那么后面的所有人都会变化，所以对于每个人的真实位置我们还需要去线段树上找，映射，我们可以直接用map来存。

方伯伯的OJ

对于操作一，修改编号 x 为 y ，并且询问排名。

首先，我们去map(关键字是编号)中找到该编号的线段树中的位置，注意不是排名，因为这个排名很有可能是错误的。然后再根据这个位置去排名线段树上找到其真实排名，并且在map上把 x 的信息去掉，把 y 的信息更新为原来 x 的信息。

方伯伯的OJ

对于操作二，把编号为 x 的点提到第一位

同样，我们还是先去map中找到编号为 x 的点在排名线段树中的位置，然后去查找，再删除，然后移动到最前面去。最后在map中更新对应的位置。

因为我们要不停的更细位置，所以为了方便根据编号来查找，并且每次插入不是在开头就是在末尾，所以这里的编号我们需要合理分配一下，最开始编号为 $[n+1, 2n]$ ，插入到最前面的点的编号依次为 $n, n-1 \cdots 1$ ，插入到后面的点的编号依次为 $2n+1, 2n+2 \cdots 3n$ ，这样排名树上的编号就是有序的了。

对于操作三，也是一样的操作。

方伯伯的OJ

对于操作4，查询排名为 k 的编号，直接在排名线段树上查询就可以了。

因为该题用户数 n 为 $1e8$ ，所以直接开权值线段树开不了，并且该题是不可以离散化的，所以我们需要用动态开点线段树来实现，然后再用一个map来实现编号对于排名的映射

电源插排

题目描述

展开

小 M 的实验室有很多电源插排。这些插排的编号从 1 到 n ，由左向右排成一排。

每天早晨，这些插排都是没有被使用的。每当一个学生来到实验室，他就将自己的笔记本电源插到某一个未被使用的插排上。

实验室的同学们都很奇怪，他们完成这个过程是这样的：首先，他们找到还没有被使用的插排的最长区间。

如果有多个区间长度相同，他们就选择最靠右的那个。然后将自己的电源插到该区间的中间。

如果区间长度是偶数，他们同样选择靠右的那个。当一个同学离开实验室时，他会将自己的电源拔出来。

数据保证每一个同学来到实验室时，至少有一个空的插排。

需要计算在区间 $[l, r]$ 已经有多少个插排被使用了。

对于 100% 的数据， $1 \leq n \leq 10^9, 1 \leq q \leq 10^5, 0 \leq k \leq 10^9, 1 \leq l \leq r \leq n$ 。

电源插排

该题其实就是一个用线段树维护最长串的问题，但是有两个问题，第一是区间很大，并且要询问最长空余区间，并且把最中间使用了，所以我们前期是不知道哪些点会被使用，所以没有办法对齐离散化。第二，我们还要知道这个同学的插头是哪一个，我们直接开一个map维护一下就可以了，其实该题和方伯伯的0J差不多，都是动态开点线段树+map映射。

可持久化数据结构

我们前面做过的数据结构题一般都是在上一次修改之后继续修改或者查询，我们只需要维护维护每一次修改完之后的数据结构的形态就可以了，也就是说我们只需要维护一棵树。

但是有些题目的问题并不是这样，他可能会在某个历史版本上做修改或者进行询问。比如：在前面第 k 个操作的基础上修改 x 的值，询问第 k 个操作后 x 这个位置的值。

那么对于这一类问题我们又该如何处理呢？

可持久化数据结构

题目描述

如题，你需要维护这样的一个长度为 N 的数组，支持如下几种操作

1. 在某个历史版本上修改某一个位置上的值
2. 访问某个历史版本上的某一位置的值

此外，每进行一次操作（对于操作2，即为生成一个完全一样的版本，不作任何改动），就会生成一个新的版本。版本编号即为当前操作的编号（从1开始编号，版本0表示初始状态数组）

输入格式

输入的第一行包含两个正整数 N, M ，分别表示数组的长度和操作的个数。

第二行包含 N 个整数，依次为初始状态下数组各位的值（依次为 $a_i, 1 \leq i \leq N$ ）。

接下来 M 行每行包含3或4个整数，代表两种操作之一（ i 为基于的历史版本号）：

1. 对于操作1，格式为 $v_i \ 1 \ loc_i \ value_i$ ，即为在版本 v_i 的基础上，将 a_{loc_i} 修改为 $value_i$
2. 对于操作2，格式为 $v_i \ 2 \ loc_i$ ，即访问版本 v_i 中的 a_{loc_i} 的值，生成一样版本的对象应为 v_i

可持久化数据结构

因为我们需要在每个历史版本上进行操作，那么很容易想到一种很暴力的做法，也就是保留每一次修改之后的树的形态，也就是维护一片森林。那么第 k 棵线段树就表示第 k 次操作之后的树的形态，那么下一次操作如果是在第 k 次操作的基础上继续操作，那么我们直接从第 k 棵树的根节点开始访问。

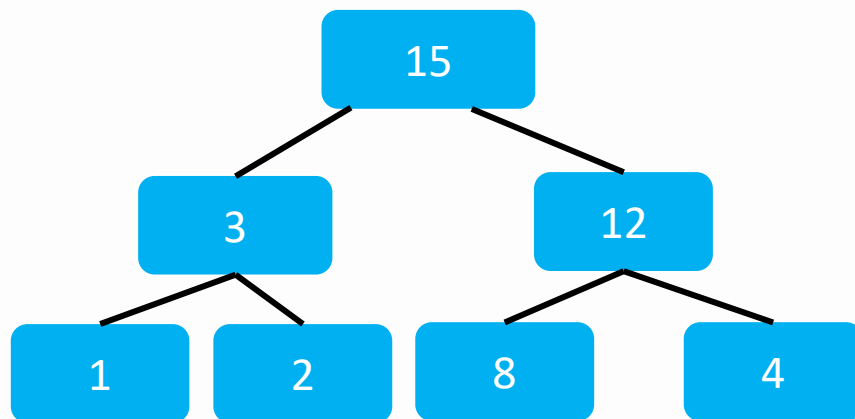
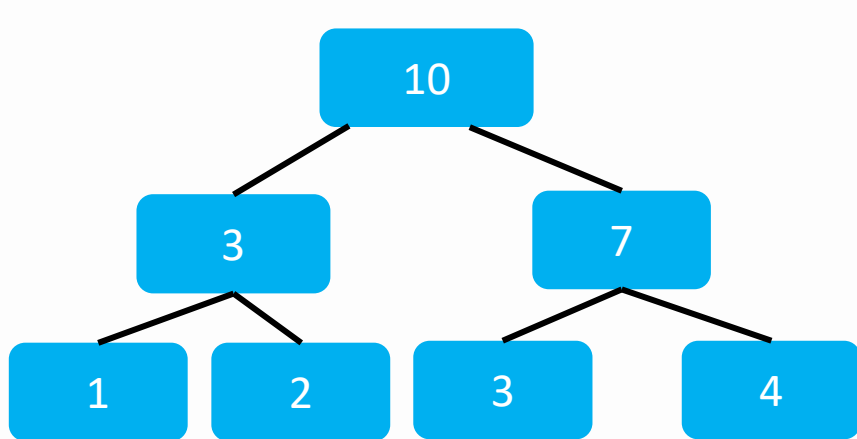
但是这样做最大的问题就是时间和空间，每一次操作都要建一棵完整的数，那么每一次操作时间复杂都是 $O(n)$ ，所以时间和空间是不满足要求的。

我们仔细观察一下相邻的两棵树

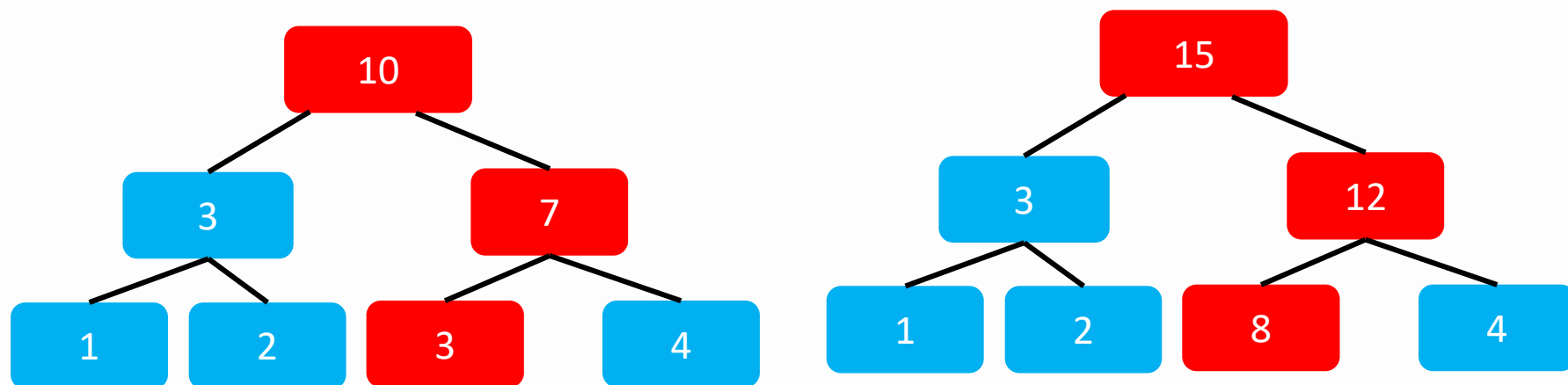
可持久化数据结构

假设我们需要用线段树来维护一次操作，那么我们来看下相邻两棵线段树的区别和联系。

原始数组 1 2 3 4 我们需要维护区间和
下一次操作：修改第3个数为8

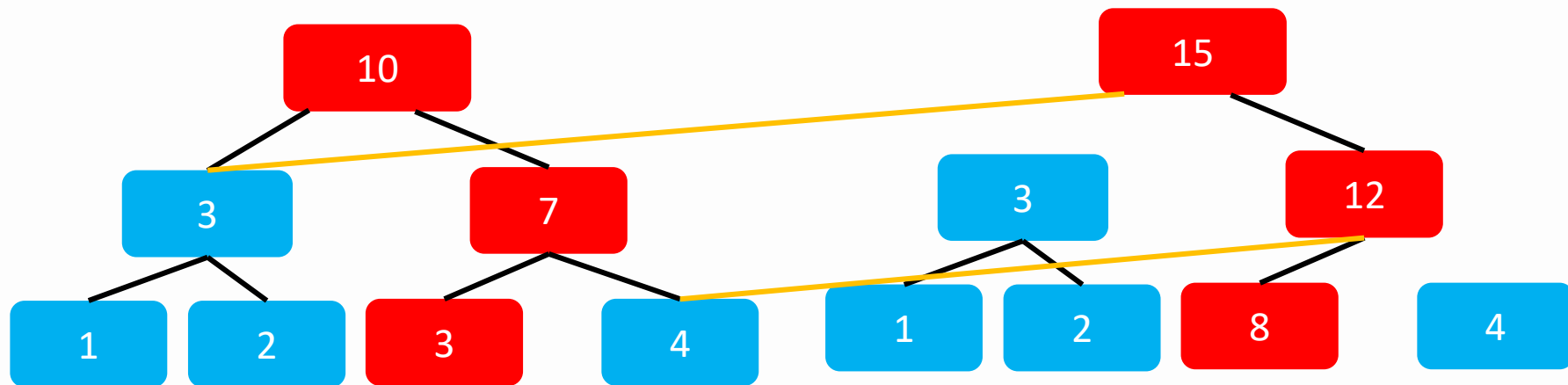


可持久化数据结构



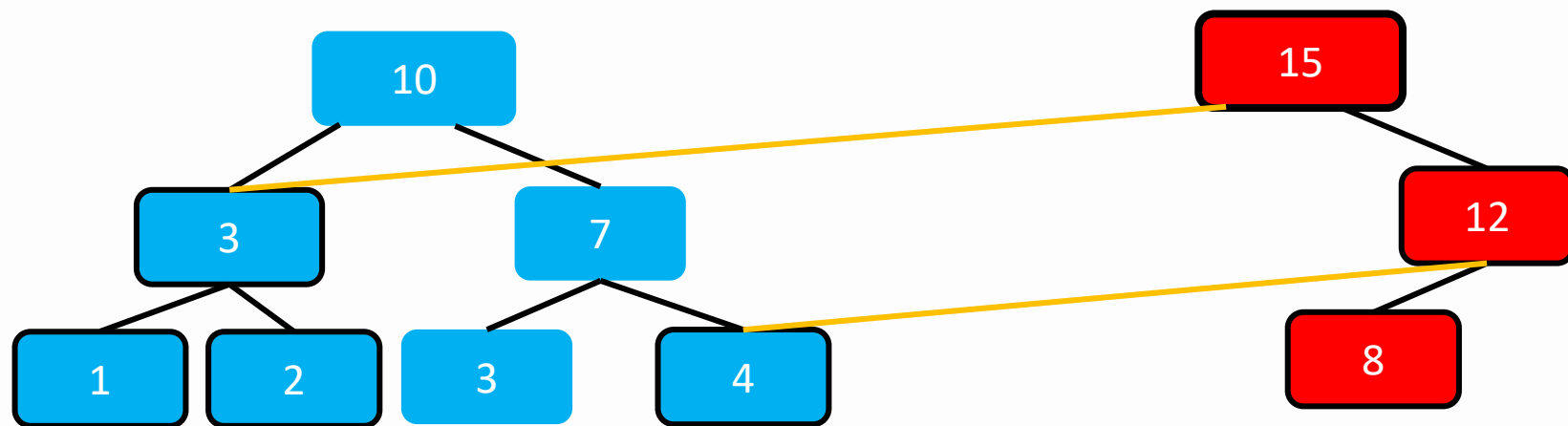
我们很容易发现，对于单点修改，每次修改前面的两颗线段树并不是完全不一样，而是有相同的部分，也有不同的部分，每次修改只是修改了一条链，即根节点到该节点所在叶子这一条链，对于其他部分，我们都是不需要修改的。所以我们并不需要重新建一棵完整的树。

可持久化数据结构



比如在该题中，我们修改第3个数的值，从根节点开始遍历，3号节点在根节点的右子树，那么根节点的左子树不需要修改，所以我们可以直接让新数的根节点的左儿子指向上一棵树的左儿子，左子树后面都不用去管了。右儿子需要修改，新开一个节点，到了右儿子后，我们发现3在他的左二子，那么右儿子直接继承，左二子新建节点。

可持久化数据结构



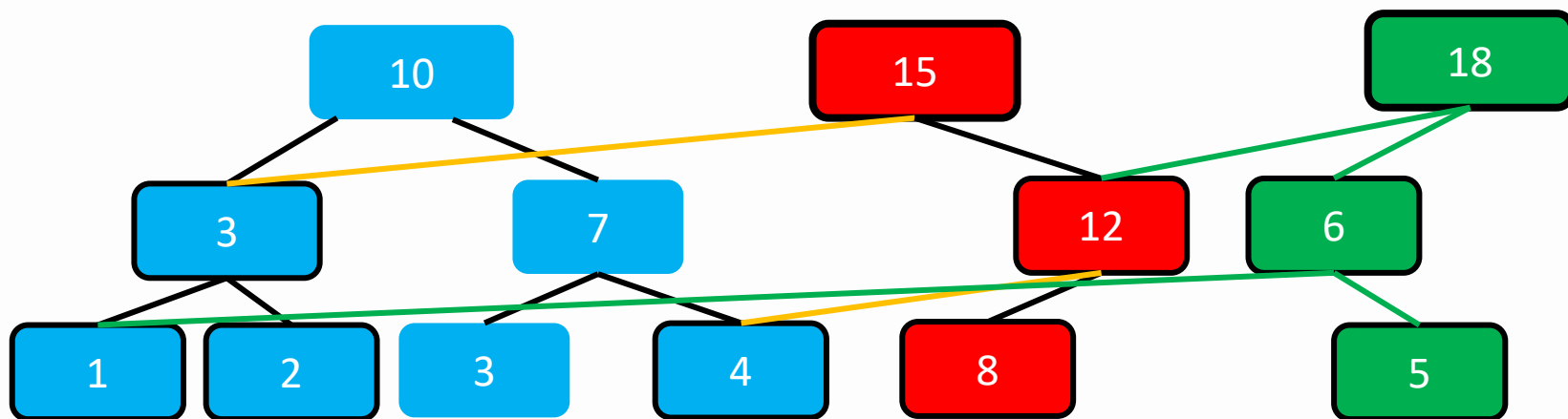
这样我们每次修改一个节点的时候，新建的节点数只有一条链的长度，也就是最多修改 $\log n$ 个节点，也只会访问 $\log n$ 个节点。图中带黑色框的部分就是新的线段树。仍然是一棵完整的线段树，但是其中的部分未修改的节点仍然是使用了前面已有的节点。

可持久化数据结构

原始数组 1 2 3 4

第一次修改 第3个数为8

第二次修改 第2个数为5



可持久化数据结构

可持久化数据结构原理特别简单，代码也不长，只是在原有的数据结构上做一个简单的修改。

空间问题：因为可持久化数据结构每次会新增一条链，即 $\lceil \log n \rceil$ 个点，那么如果对于一个长度为 n 的数组，有 n 次操作，那么 n 次操作完之后的点的个数为 $4*n + n*\log n$ ，对于10w个点，一般数组开20+就可以了，100w开30，一般来说为了不出问题，一般都开30倍。

可持久化数据结构只能采用动态开点的形式，如果采用 $id*2$ 的形式，肯定爆空间。

可持久化数据结构

```
int build(int l,int r)//动态建树
{
    int now=++k;
    if(l==r)
    {
        tree[now].val=a[l];
        return now;
    }
    int mid=(l+r)/2;
    tree[now].l=build(l,mid);
    tree[now].r=build(mid+1,r);
    return now;//把值返回给父亲节点
}
root[0]=build(1,n);
```


可持久化数据结构

```
for(int i=1;i<=m;i++)
{
    scanf("%d%d",&old,&opt);//old表示历史版本
    if(opt==1)//修改
    {
        scanf("%d%d",&x,&y);
        root[i]=update(root[old],1,n,x,y);
    }
    else if(opt==2)//查询
    {
        scanf("%d",&x);
        printf("%d\n",query(root[old],1,n,x));
        root[i]=root[old];//查询要继承历史版本
    }
}
```

可持久化数据结构

```
int update(int pre,int l,int r,int x,int y)
{//两颗线段树同时跳转，始终保持在相同区间，这样才能继承
    int now=++k;
    tree[now].l=tree[pre].l;
    tree[now].r=tree[pre].r;//区间相同
    if(l==r)
    {
        tree[now].val=y;
        return now;//边更新边建树，所以要返回值
    }
    int mid=(l+r)/2;
    if(x<=mid)
        tree[now].l=update(tree[pre].l,l,mid,x,y);
    else
        tree[now].r=update(tree[pre].r,mid+1,r,x,y);
    return now;
}
```

可持久化数据结构

```
int query(int pre,int l,int r,int x)
{ // 在历史版本上查询, 和普通线段树的查询一样
    if(l==r)
        return tree[pre].val;
    int mid=(l+r)/2;
    if(x<=mid)
        return query(tree[pre].l,l,mid,x);
    else
        return query(tree[pre].r,mid+1,r,x);
}
```



可持久化数据结构

像这种询问历史版本的数据结构的题特别少，但是可持久化数据结构除了可以做这种很明显的带有历史版本的数据结构的题之外，还有一个很常见的应用——询问静态区间第 k 大。



区间第K大

给定N个整数构成的序列，将对于指定的闭区间查询其区间内的第K小值。

输入：

5 5

25957 6405 15770 26287 26465

2 2 1

3 4 1

4 5 1

1 2 2

4 4 1

输出：

6405

15770

26287

25957

26287

区间第K大

我们以前做的题都是用线段树求解区间极大(小)值，最多也就是次大(小)值，但是对于第K大的值，我们还是第一次遇到。因为K是不确定的，所以线段树肯定维护不了，那我们考虑一下平衡树是否可以实现。

平衡树建树要么依据区间编号建树，要么依据权值大小建树。如果我们保证区间有序，那么查询的区间是很容易分离出来，但是区间内的答案却不是按照权值排序的；如果我们保证权值有序，那么我们就没有办法把需要查询的区间分离出来，自然也就没有办法求解区间第K大。

所以我们需要学习一种新的数据结构来解决问题。

前缀第K大

我们先思考一个简单的题：依次给定 n 个数，再给定 m 个询问，问前 i 个数中第 k 大的数是多少，保证每次询问的位置递增。

输入：

1 2 3 4 5

3

2 2

4 2

5 3

输出：

1

3

3

前缀第K大

当然，这道理可以用我们学过的平衡树很容易解决，但是实际上连平衡树都不需要，一棵线段树就可以解决了。

我们可以用权值线段树来解决、先建一颗 n 个结点的空树，对于查询前 i 个数第 k 大，我们可以先把前 i 个数的权值当做编号放进去，每个结点记录当前子树有多少个数，在查询的时候，参考平衡树的查询第 k 大的方式。

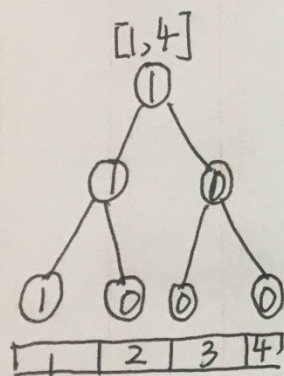
主席树

所以我们知道如果是前缀第 k 大，我们可以用权值线段树来查询，但是如果是区间呢？如果我们需要知道每次查询的区间内到底有哪些数，那我们就需要可持久化数据结构来维护，简单来说，也就是如果我们还是用线段树来维护，那么我们需要知道每次插入一个数之后的线段树的形态。这样我们就可以用两个不同时间段的线段树相减就得到了我们需要的区间。因为相减之后还存在的数就表示在该区间内出现的数。

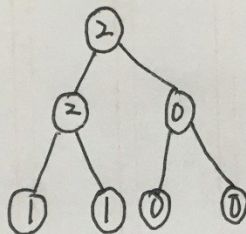
主席树

1 2 3 4

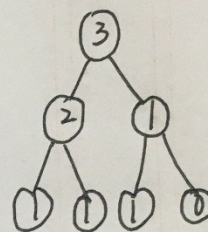
1. 插入 1.



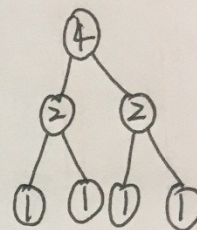
2. 插入 2



3. 插入 3.

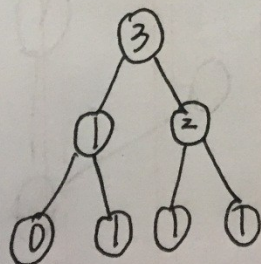


4. 插入 4.



查询区间 $[2, 4]$ 先找到第一棵树 再找到第 4 棵树 相减.

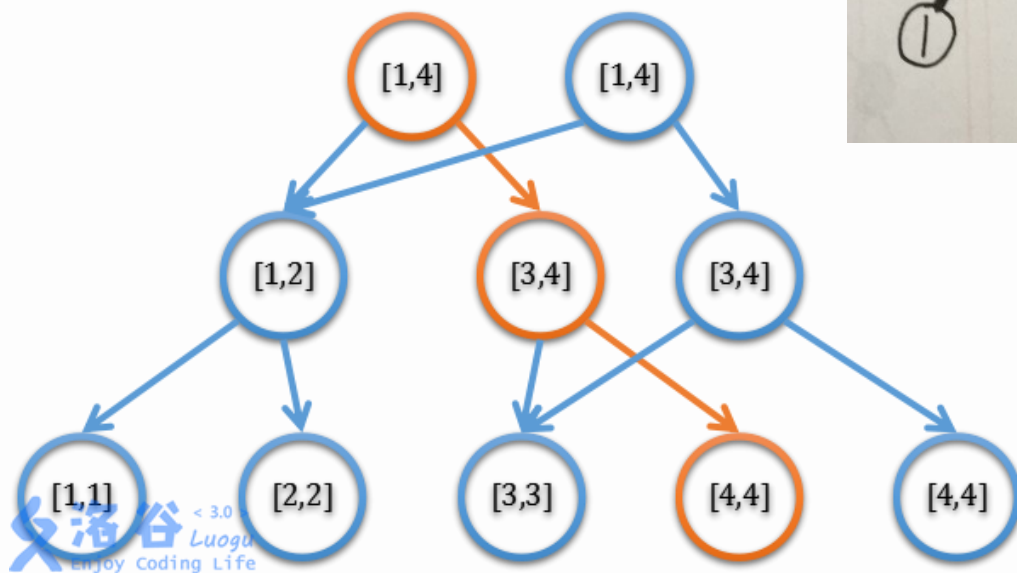
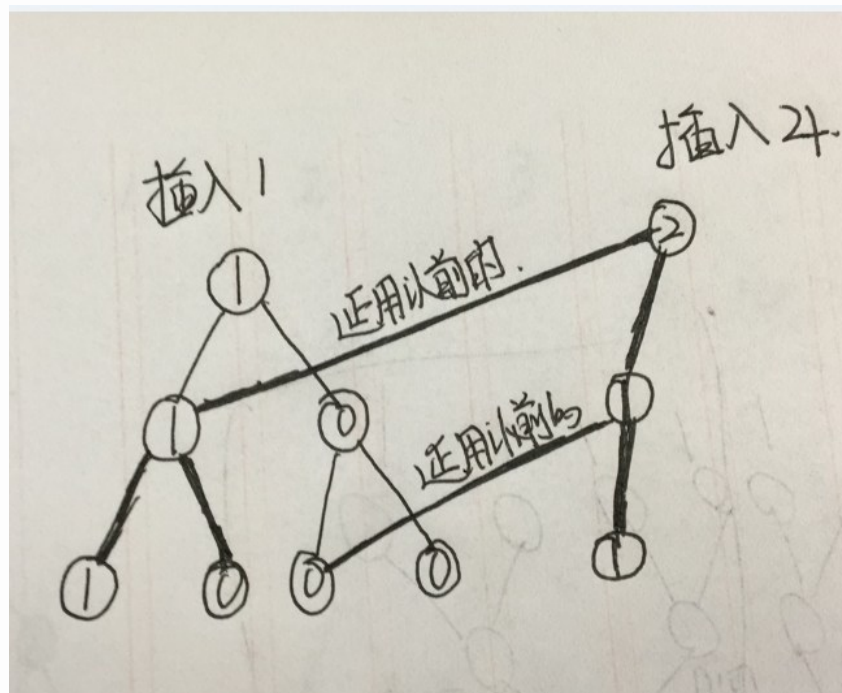
相减之后树的结构即为区间 $[2, 4]$ 出现的数.



主席树

这样做是可以解决问题，但是这样做的时间复杂度和空间复杂度都是我们不能接受的，因为对于 n 个数来说，需要开 n 棵线段树，时间复杂度和空间复杂度都是 $O(n^2)$ 的，所以我们需要优化。我们仔细观察一下相邻的两棵树，我们会发现，相邻的两棵树只有一条链不相同，即修改的那个节点向上的那条链，其他地方都是一样的。所以我们没有必要再开一颗完整的线段树，只需要在上一棵树的基础上重新增加一条链就可以了，剩下的不变的地方就沿用以前的就可以了。这样空间复杂度只需要 $N+N\log N$ ，对于100000的数据， $*1 \ll 5$ 就够了，时间复杂度也是 $N\log N$ 。

主席树



实现过程

第一步：因为我们要建权值线段树，但是数的范围比较大，所以我们需要先离散化，以免空间不够。注意需要因为要知道原来的值，所以开双向映射。

第二步：初始化建树，先建一棵空的线段树，为了节省空间已经后期好更改左右儿子的指针，所以主席树建树采用线段树动态开点来实现。因为主席树实际上一片森林而不是一棵树，所以我们需要知道每一棵树的根结点，所以开一个root数组来记录每次插入数之后的根结点新生成的树的根结点的编号。

实现过程

```
int build(int l,int r)
{
    int root=++k; //动态开点
    tree[root].sum=0; //目前是空树
    if(l<r)
    {
        int mid=(l+r)/2;
        tree[root].l=build(l,mid);
        //左右子树开好点之后再把该点编号返回给指向他的结点
        tree[root].r=build(mid+1,r);
    }
    return root;
}
```


实现过程

第三步：把第 i 个数当做第 i 次插入的数，分别在上一棵树的基础上建立 n 个线段树，注意空间需要开 $n \log n$

```
int updata(int pre,int l,int r,int x)//pre表示上一棵树的根结点
{
    int root=++k;
    tree[root].l=tree[pre].l;//每课线段树的区间范围是一样的
    tree[root].r=tree[pre].r;
    tree[root].sum=tree[pre].sum+1;//比上一次多了一个数
    if(l<r)
    {
        int mid=(l+r)/2;
        if(x<=mid)//如果新增节点在左边，则右边子树不变
            tree[root].l=updata(tree[pre].l,l,mid,x);
        else
            tree[root].r=updata(tree[pre].r,mid+1,r,x);
    }
    return root;
}
```

实现过程

第四步：查询区间 $[l, r]$ 第K大的数，即用第r棵线段树的每个根结点-第 $(l-1)$ 棵线段树的每个根结点，这样得到的树的形态就是区间 $[l, r]$ 之间的数组成的线段树。再按照权值线段树的方法分别根据左右区间的数的数量来找到第K大的值是多少。

```
int query(int u,int v,int l,int r,int ki)
{ //u表示(l-1),v表示r
  if(l>=r)//表示找到了第ki大的数
    return l;
  int x=tree[tree[v].l].sum-tree[tree[u].l].sum;
  int mid=(l+r)/2;
  if(x>=ki)//表示在右边区间
    return query(tree[u].l,tree[v].l,l,mid,ki);
  else
    return query(tree[u].r,tree[v].r,mid+1,r,ki-x);
}
```


可持久化并查集

我们以前学习的并查集只可以合并而不可以撤销，如果需要维护一个既可以合并又可以撤销的的并查集可以用可持久化并查集来维护。

在并查集的合并操作中有一个比较重要的操作叫路径压缩，实际上我们的并查集可以看做一棵树，但是如果使用了路径压缩，这棵树的形态就会发生改变，不仅仅只是左右子树中之一会变化。所以可持久化并查集不能使用路径压缩。

那么为了保证并查集的效率，我们应该如何合并呢？其实并查集的效率 and 并查集的深度有关，只要保证合并之后并查集的深度最小就可以了，所以我们可以用启发式合并(按秩合并)，即把深度小的并查集的根结点，连接到深度大的并查集的根结点去。

可持久化并查集

```
int f1=getf(x);
int f2=getf(y); //求祖先
if(deep[f1]>deep[f2])
{
    int tmp=f1;f1=f2;f2=tmp;
}
fa[f1]=f2; //把f1合并到f2
if(deep[f1]==deep[f2])
    deep[f2]++; //相等, 深度+1, 否则深度不变
```

可持久化并查集

可持久化并查集可以用可持久化线段树来实现，对于线段树的每一个位置，我们存放该结点所在集合的根结点的信息。

每次并查集合并的时候，我们只需要把其中一个集合的根结点的父亲结点更新为另一个集合的父亲结点就可以了。和前面线段树的操作一样，对从根结点到这个叶子结点这一条链新开一串结点来更新其父亲结点。更新完父亲结点之后，如果这两个集合本来深度就相同，那么还需要更新一下 f_2 所在集合的深度。

在并查集中求祖先实际上就是线段树的查询操作，在线段树上找到其祖先，如果是集合的根，就不再找了，如果不是，那么继续在线段树上找，直到满足条件为止。

可持久化并查集

```
int build(int l,int r)
{
    int now=++k;
    if(l==r)
    {
        tree[now].fa=l; // 最开始父亲结点都是它本身
        return now;
    }
    int mid=(l+r)/2;
    tree[now].l=build(l,mid);
    tree[now].r=build(mid+1,r);
    return now;
}
```

可持久化并查集

```
int update(int pre,int l,int r,int x,int y)
{
    int now=++k;
    tree[now]=tree[pre];
    if(l==r)
    {
        tree[now].fa=y; //更新该点的父亲结点
        deep[now]=deep[pre]; //先继承根结点深度
        return now;
    }
    int mid=(l+r)/2;
    if(x<=mid)
        tree[now].l=update(tree[pre].l,l,mid,x,y);
    else
        tree[now].r=update(tree[pre].r,mid+1,r,x,y);
    return now;
}
```

```
root[i]=update(root[i-1],1,n,tree[f1].fa,tree[f2].fa);
```

可持久化并查集

```
int getf(int now,int x)
{
    int fx=query(now,1,n,x);
    if(x==tree[fx].fa)
        return fx;
    else
        return getf(now,tree[fx].fa);
}

int f1=getf(root[i-1],x);
int f2=getf(root[i-1],y);
if(tree[f1].fa==tree[f2].fa)
{
    root[i]=root[i-1];
    continue;
}
if(deep[f1]>deep[f2])
{
    int tmp=f1;f1=f2;f2=tmp;
}
root[i]=update(root[i-1],1,n,tree[f1].fa,tree[f2].fa);
if(deep[f1]==deep[f2])
    add(root[i],1,n,tree[f2].fa);
```

可持久化并查集

```
void add(int now,int l,int r,int x)
{
    //当两个并查集深度一样的时候, f2所在并查集深度+1
    if(l==r)//在前面更新父亲结点的时候结点已经添加了
    {
        deep[now]++;
        return ;
    }
    int mid=(l+r)/2;
    if(x<=mid)
        add(tree[now].l,l,mid,x);
    else
        add(tree[now].r,mid+1,r,x);
}
```



练习题

洛谷	3960	列队
洛谷	3285	方伯伯的OJ
洛谷	3968	教室插排
洛谷	3834	可持久化线段树2(主席树)
洛谷	3402	可持久化并查集
洛谷	3168	任务查询系统

